

Predictive maintenance: metrics, leakages, boosting

UE ISD-1020 · Machine Learning / Deep Learning

Stéphane Rivaud

Université Paris-Saclay · Master ISD (apprenticeship)

21 September 2026

Where we are

- **UE ISD-1020** “Machine Learning / Deep Learning”
- Master Informatique, **ISD** track, apprenticeship
- Université Paris-Saclay · **PUIO B210**
- Teaching team: Stéphane Rivaud, Solal Nathan and Dylan Séchet
- Today: **09:00–12:30** (afternoon 13:30–17:00)

D0 · Telco
churn, 26.5% Yes



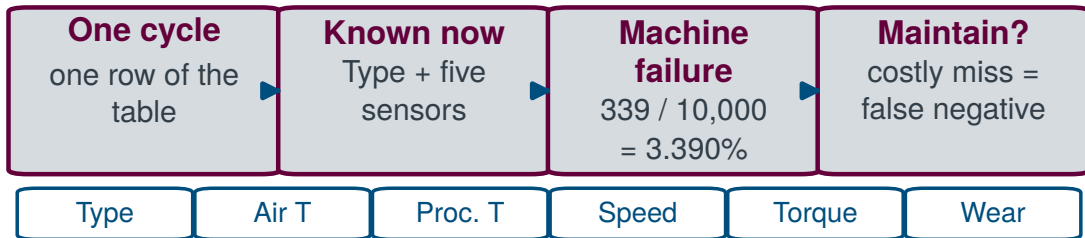
D1 · AI4I
failures, 3.390%

Same room, same protocol, new table.

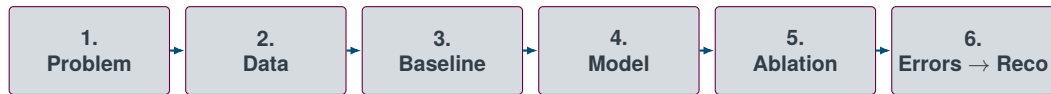
Which machine will fail

Definition

We must decide, for each cycle, whether to schedule maintenance. The target is $y = \text{Machine failure} \in \{0, 1\}$. The costly error is a missed failure (a false negative).



This morning's thread



Metrics for 3.4% positives → two leakages → boosting → errors by Type → the shared notebook.

Three percent, and accuracy lies

Definition

Accuracy = $(TN + TP)/n$. **Recall** = $TP/(TP + FN)$. Always 0: accuracy 0.966, recall 0 — 97% that catches nothing.

Always 0 · acc. 96.6%

True	no failure	1,932	0	True	
	failure	68	0		
		no failure	failure		
		Predicted			

% Logistic · acc. 97.5%

no failure	1,930	2
failure	49	19
	no failure	failure
	Predicted	

Four cells, four costs

Definition

A confusion matrix at one threshold (here 0.5). Pedagogical costs: $FN \times 100$, $FP \times 1$.
Logistic: 49 missed failures, accuracy still 0.975.

Logistic at 0.5 · cost 4,902 (always 0: 6,800)

True	no failure	failure
	no failure	failure
no failure	1,930 TN	2 FP · ×1
failure	49 FN · ×100	19 TP

Four cells, four costs

Definition

Precision = $TP / (TP + FP)$. **Recall** = $TP / (TP + FN)$. **F1** is their harmonic mean, at **one** threshold. Logistic at 0.5: P 0.905 · R 0.279 · F1 0.427.

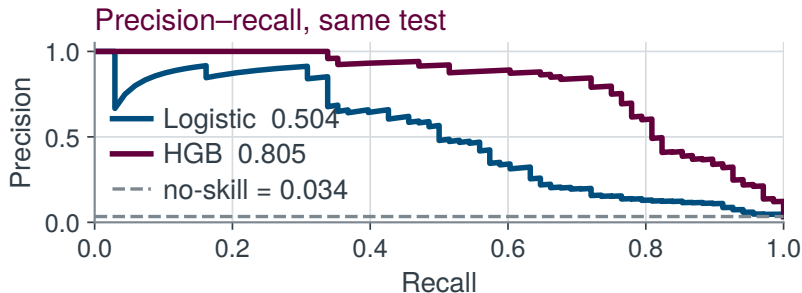
Logistic at 0.5 · cost 4,902 (always 0: 6,800)

True	no failure	1,930 TN	2 FP · ×1
	failure	49 FN · ×100	19 TP
		no failure	failure
		Predicted	

PR-AUC, not ROC

Definition

Average precision is the area under the precision–recall curve. It ranks failures. No-skill = $68/2,000 = 0.034$. Logistic 0.504 · HGB 0.805. ROC-AUC is too kind to the 0s.



A cost matrix

Definition

Expected cost = $100 \cdot \text{FN} + 1 \cdot \text{FP}$ on this test, at threshold 0.5. The 100/1 pair is pedagogical, not a factory invoice.

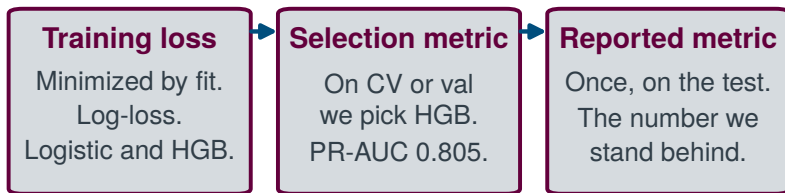
Policy	FN	FP	Cost
Always 0	68	0	6,800
Logistic	49	2	4,902
HGB	23	7	2,307

Threshold 0.5 is not magic: we set it on the true cost.

Loss is not the selection metric

Definition

Training loss (log-loss) is what `fit` minimizes. The **selection metric** (PR-AUC on CV or val) chooses among candidates. The **reported metric** is the single hold-out test score.

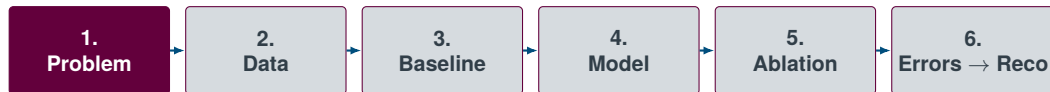


We minimize one function; we choose and report another.

The selection metric belongs to the problem

Definition

Protocol step 1 names the decision, the target, and the **selection metric**, before any model family. Step 4 names the family and the **training loss** that fit descends. They are not siblings.



Changing the model without the metric is changing the subject.

One split is not enough

Definition

The hold-out (2,000 rows, 68 failures) is one draw. Logistic PR-AUC on 5 train folds: $0.46 \cdot 0.41 \cdot 0.39 \cdot 0.50 \cdot 0.43 \rightarrow 0.438 \pm 0.040$. Stratify: $\sim 3.4\%$ in every fold.

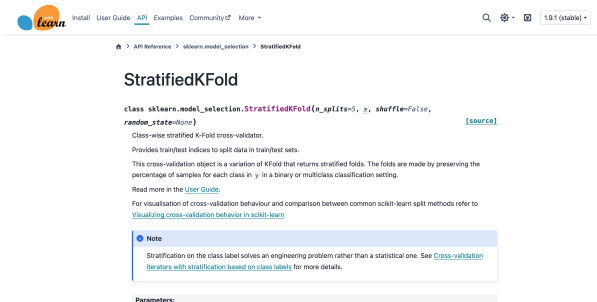
Train · 8,000 rows · 5 folds

Hold-out test

fold 1	fold 2	CV test	fold 4	fold 5	untouched
54 / 1600 3.4%	54 / 1600 3.4%	54 / 1600 3.4%	54 / 1600 3.4%	55 / 1600 3.4%	68 / 2000 3.4%
AP 0.46	AP 0.41	AP 0.39	AP 0.50	AP 0.43	

Definition

Five stratified folds; `cross_val_score` clones the Pipeline; scoring = average_precision. The 20% test stays untouched for the number we cite.



The screenshot shows the scikit-learn documentation for `StratifiedKFold`. At the top, there's a navigation bar with links for 'Install', 'User Guide', 'API', 'Examples', 'Community', and 'More'. The 'API' link is active. Below the navigation bar, the breadcrumb trail reads 'API Reference > sklearn.model_selection > StratifiedKFold'. The main heading is 'StratifiedKFold'. Below it, the class signature is shown: `class sklearn.model_selection.StratifiedKFold(n_splits=5, *, shuffle=False, random_state=None)`. A '[source]' link is provided. The description states: 'Class-wise stratified K-Fold cross-validator. Provides train/test indices to split data in train/test sets. This cross-validation object is a variation of KFold that returns stratified folds. The folds are made by preserving the percentage of samples for each class in y in a binary or multiclass classification setting. Read more in the User Guide. For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to Visualizing cross-validation behavior in scikit-learn'. A 'Note' box contains the text: 'Stratification on the class label solves an engineering problem rather than a statistical one. See Cross-validation iterators with stratification based on class labels for more details.' At the bottom, there is a 'Parameters:' section.

```
class sklearn.model_selection.StratifiedKFold(n_splits=5, *, shuffle=False,
random_state=None)
[source]
Class-wise stratified K-Fold cross-validator.
Provides train/test indices to split data in train/test sets.
This cross-validation object is a variation of KFold that returns stratified folds. The folds are made by preserving the
percentage of samples for each class in y in a binary or multiclass classification setting.
Read more in the User Guide.
For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to
Visualizing cross-validation behavior in scikit-learn
```

Note

Stratification on the class label solves an engineering problem rather than a statistical one. See [Cross-validation iterators with stratification based on class labels](#) for more details.

Parameters:

`sklearn.model_selection.StratifiedKFold`

Silent leakage: the scaler

Definition

Leak: the test took part in fit (mean and scale of all X). PR-AUC 0.505 versus 0.504 — **silent**, not absent. Remedy: the scaler lives inside the Pipeline.

Wrong

Fit scaler on all X
then split

PR-AUC 0.505

Right

Split first
Pipeline fits train only

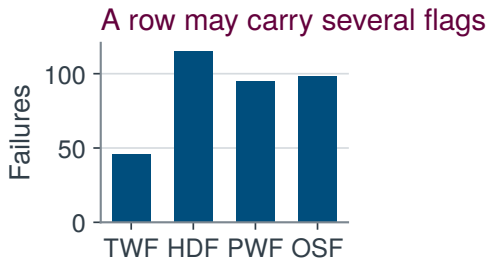
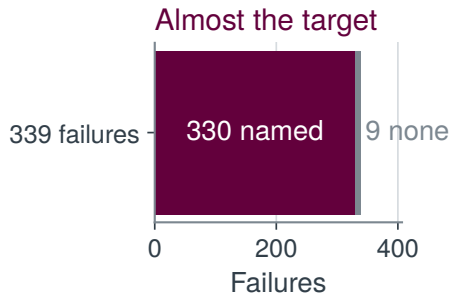
PR-AUC 0.504

silent $\neq$ absent

Noisy leakage: the sub-flags

Definition

TWF, HDF, PWF, OSF, RNF are post-failure diagnostics. Put them in X : recall 0.941, PR-AUC 0.945 — production does not have these columns.



The Pipeline is the remedy

Definition

A ColumnTransformer (scaler on the sensors, one-hot on Type) sits inside a Pipeline. Fit only on train. An LLM that scales all X then splits: it runs, and it leaks.



Install User Guide API Examples Community 12 More

1.0.1 (stable)

API Reference > sklearn.compose > ColumnTransformer

ColumnTransformer

```
class sklearn.compose.ColumnTransformer(transformers, *, remainder='drop',
sparse_threshold=0.3, n_jobs=None, transformer_weights=None, verbose=False,
verbose_feature_names_out=True)
```

[\[source\]](#)

Applies transformers to columns of an array or pandas DataFrame.

This estimator allows different columns or column subsets of the input to be transformed separately and the features generated by each transformer will be concatenated to form a single feature space. This is useful for heterogeneous or columnar data, to combine several feature extraction mechanisms or transformations into a single transformer.

Read more in the [User Guide](#).

Added in version 0.20.

Parameters:

transformers : list of tuples

List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data.

name : str

When the Pipeline is used for prediction, this allows the transformers used the estimator to be identified. [\[sklearn.org\]](#)



Install User Guide API Examples Community 12 More

1.0.1 (stable)

API Reference > sklearn.pipeline > Pipeline

Pipeline

```
class sklearn.pipeline.Pipeline(steps, *, transform_input=None, memory=None,
verbose=False)
```

[\[source\]](#)

A sequence of data transformers with an optional final predictor.

[Pipeline](#) allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final [predictor](#) for predictive modeling.

Intermediate steps of the pipeline must be transformers, that is, they must implement [fit](#) and [transform](#) methods.

The final [estimator](#) only needs to implement [fit](#). The transformers in the pipeline can be cached using [memory](#) argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a `__`, as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to `'passthrough'` or `None`.

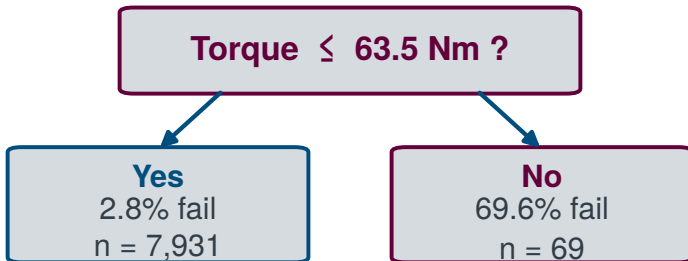
For an example use case of [Pipelines](#) combined with [GridSearchCV](#), refer to [Selecting dimensionality reduction with Pipeline and GridSearchCV](#). The example [Spelling: chaining a PCA and a logistic regression](#) shows how to grid search on a pipeline using `'__'` as a separator in the parameter names.

Read more in the [User Guide](#).

A tree: one threshold at a time

Definition

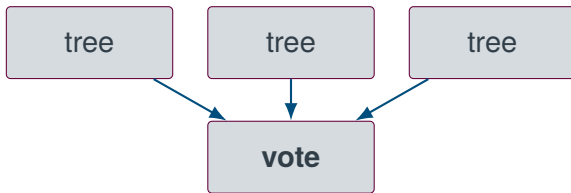
A decision tree is nested axis-aligned splits. A stump on this train asks “Torque ≤ 63.5 Nm?” then stops: 2.8% vs 69.6% failure.



A forest: the vote

Definition

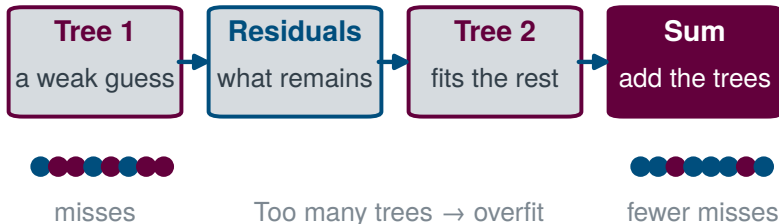
Bagging: many trees, each on a draw of rows and features. We average the votes — a weird tree is drowned. Variance falls; we are not yet chasing residuals.



Boosting: chase the residuals

Definition

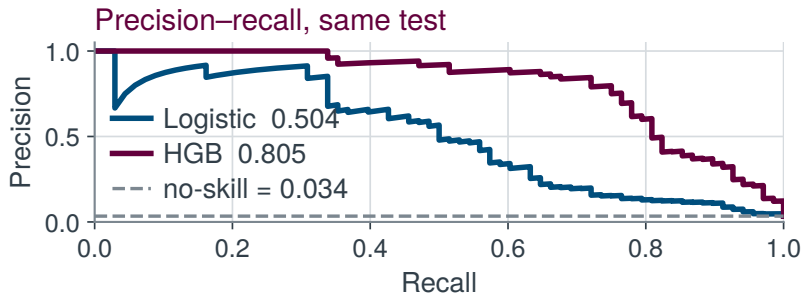
Start from a weak learner. The next tree fits what remains — the **residuals**. Add: $\text{prediction} \leftarrow \text{prediction} + \text{tree}$. More trees: more capacity. Too many trees: overfit.



HistGradientBoosting

Definition

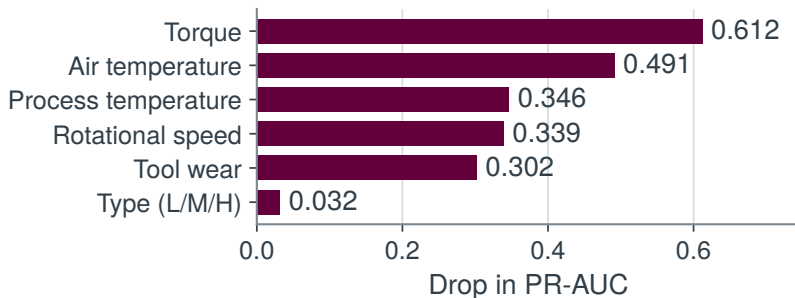
HGB = boosting + histogram splits; max_iter=80. Test: recall 0.662 · F1 0.750 · PR-AUC **0.805**. Train CV 0.804 ± 0.017 — the tabular baseline to beat.



Importance by permutation

Definition

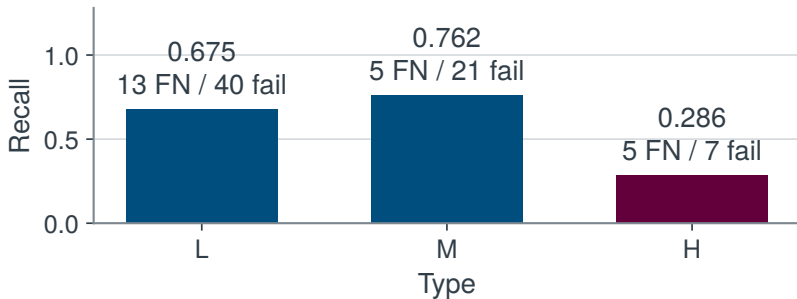
Shuffle column j on the test; the drop in PR-AUC is the importance of j for **this** model, not a physical cause. Torque 0.612. Five repeats are enough for a session diagnosis.



Read the errors by Type

Definition

Global HGB recall 0.662 hides Type H at 0.286 (5 FN / 7 failures). An alert, not a proof.



Toward a recommendation

On this test

HGB beats logistic and always-0 on PR-AUC and on the relative cost. Do not deploy a model that saw the sub-flags.

Before a unique threshold

Set it on the true factory cost (100 / 1 is only an example). Collect more H failures — or a threshold per Type.

This morning's lab

1. Shared course folder / link at the start of the session
2. File → Save a copy in Drive
3. Default **autonomous**; a hinted variant if the room struggles
4. Thread: metrics → Pipeline → two leakages → CV → HGB → errors
5. Six **protocol** cells at the bottom (oral exam of 5 October)

Attributions and licences

Module material: Stéphane Rivaud.

Tutorials

- scikit-learn MOOC · modules 1, 2, 7 · CC BY 4.0
- PyTorch tutorials · BSD-3-Clause
- d2l.ai · CC BY-SA 4.0

Datasets

- AI4I 2020 / UCI 601 · CC BY 4.0
- Telco IBM · Apache-2.0
- éCO2mix · Licence Ouverte v2.0

Documentation

- scikit-learn · BSD-3-Clause
- PyTorch · BSD-3-Clause

scikit-learn and PyTorch marks remain with their owners; used here only to identify the tools.